Geavanceerde Computerarchitectuur Labo 4
Coalesced memory access and device (GPU) memory types

Arno De Lathauwer
Indra Coppens

Maart 2025

1 Context

In dit labo onderzoeken we de invloed van GPU-geheugengebruik op de prestaties van CUDA-kernels. Eerst vergelijken we coalesced memory access met uncoalesced memory access door twee kernels te implementeren die de rode kleurcomponent van een afbeelding inverteren. Het verschil tussen beide kernels is dat de afbeeldingen een andere geheugenindeling hebben, namelijk RGBRGB... (uncoalesced) en RRR...GGG...BBB... (coalesced). Vervolgens analyseren we de impact van global, shared en constant memory op de rekentijd van matrixvermenigvuldiging.

2 Methodologie

2.1 Part 1: Coalesced memory access

In dit deel analyseren we het effect van coalesced en uncoalesced memory access op de rekentijd van een CUDA-kernel. Dit doen we door een afbeelding in te laden (in de vorm van een flattened array), de rode kleurcomponent te inverteren en het resultaat op te slaan. Er worden twee geheugenindelingen vergeleken:

- Uncoalesced memory access: de flattened array heeft als formaat `RGBRGBRGB...`, hierbij worden de rode pixelwaarden (waarop een bewerking moet worden uitgevoerd) niet op opeenvolgende geheugenadressen opgeslagen. De bijhorende code is te vinden in Appendix A.
- Coalesced memory access: de flattened array heeft als formaat `RRR...GGG...BBB...`, waarbij de rode pixelwaarden wel op opeenvolgende adressen in het geheugen worden opgeslagen. De code is beschikbaar in Appendix B

We implementeren een CUDA-kernel die de rode kleurcomponent van elke pixel inverteerd ($R = 255 - R$) voor beide types geheugenindeling. Bij het formaat RGBRGBRGB... wil dit zeggen dat de pixelwaarden met index deelbaar door drie ($index \ \% \ 3 == 0$) worden aangepast en er niets gebeurt met de andere pixelwaarden. De pixelwaarden die geaccessed worden in het geheugen (om te bewerken) zitten dus niet op opeenvolgende adressen in het geheugen. In het andere geval (formaat RRR...GGG...BBB...) worden de eerste derde van de pixelwaarden bewerkt. Na deze bewerking wordt de array terug naar de CPU gekopieerd en opgeslagen als nieuwe PPM-afbeelding. Doordat bij het eerste formaat de nodige elementen niet naast elkaar liggen, en er dus uncoalesced memory access is, verwachten we dat deze implementatie trager is: bij uncoalesced memory access moeten de rode pixelwaarden één voor één worden opgehaald uit het geheugen, bij coalesced memory access kunnen alle nodige pixelwaarden in één keer opgehaald worden.

Om de prestaties van beide kernels te evalueren, wordt de uitvoertijden vergeleken tussen beide formaten. Daarnaast wordt ook de invloed van verschillende afbeeldingsgroottes op de uitvoeringstijd bekeken.

2.2 Part 2: Global, shared and constant memory

In dit deel analyseren we hoe het gebruik van verschillende types geheugen in CUDA de prestaties van een matrixvermenigvuldiging beïnvloeden. We implementeren drie versies van een matrixvermenigvuldigingskernel die gebruik maakt van repsectievelijk: global memory, shared memory en constant memory. Elke kernel berekent de vermenigvuldiging van twee matrices $C = A \times B$. We meten de rekentijden en vergelijken deze voor de verschillende geheugentypes. Daarnaast zullen we ook de invloed van de inputgrootte op de rekentijden bekijken en kijken hoelang het duurt om het geheugen te migreren: van de CPU naar het globale GPU geheugen en van het globale GPU geheugen naar het shared GPU geheugen.

2.2.1 Global memory

In deze kernel worden alle bewerkingen uitgevoerd op data die rechtstreeks uit global memory wordt opgehaald en ernaar wordt weggeschreven. We kopiëren beide matrices naar het globale geheugen van de GPU. Daarna berekent elke thread één element van matrix C door een rij uit matrix A en een kolom uit matrix B te vermenigvuldigen en te sommeren.

$$C_{ij} = \sum_{k=0}^{n} A_{ik} \cdot B_{kj}$$

Om de rij en kolom te bepalen waarop deze thread zijn bewerking moet uitvoeren hebben we de index van de thread nodig. Deze index wordt gedeeld door het aantal kolommen van de oplossing om de rij-index te bekomen (*int row = idx / width_solution*). Analoog wordt de kolom-index bekomen door de modulo van de index met dit aantal kolommen te nemen (*int col = idx % width_solution*). Tot slot wordt het resultaat terug van de GPU naar de CPU gekopieerd. Dit resulteert in een eenvoudige, maar relatief trage uitvoering. De code is te vinden in Appendix C.

2.2.2 Shared memory

Om de efficiëntie te verhogen, implementeren we een tweede kernel waarbij threads data eerst van global memory naar shared memory kopiëren. Als eerste stap zal elke thread één element van zowel A als B kopiëren naar shared memory. Dit kunnen ook meerdere elementen zijn indien we werken met meerdere blokken, maar om het concept goed te vatten werken we hier met één blok. Daarna zullen we de threads synchroniseren (met de `__syncthreads()`-functie) om ervoor te zorgen dat er geen enkele thread al begint met bewerkingen uit te voeren alvorens alle elementen in shared memory zitten. Vervolgens wordt de vermenigvuldiging van de twee matrices uitgevoerd, dit gebeurt volledig analoog als bij het globale geheugen. Deze aanpak vermindert de hoeveelheid dure global memory-toegangen, wat de rekentijd verlaagt. De bijhorende code is te vinden in Appendix D.

2.2.3 Constant memory

Tot slot maken we dezelfde implementatie door gebruik te maken van constant memory. Constant memory is read-only, en kan nog snellere resultaten behalen dan shared memory in specifieke use cases: namelijk, wanneer meerdere threads steeds dezelfde data lezen. In dit geval wordt als eerste de matrices gekopieerd van de CPU naar het constant memory van de GPU, dit gebeurd met de functie `cudaMemcpyToSymbol`. Daarna gebeurd alles volledig analoog als bij het globale geheugen. De relevante code is vermeld in Appendix E.

3 Observaties/resultaten

3.1 Part 1: Coalesced memory access

In dit experiment wordt de uitvoeringstijd vergeleken tussen coalesced en uncoalesced geheugenaccess, met verschillende invoer- en gridgroottes. Bij beide implementaties werden de rode pixelwaarden correct geinverteerd. De coalesced kernel biedt ehcter een aanzienlijke snelheidswinst ten opzichte van de uncoalesced kernel, wat duidelijk te zien is in *Figuur 1* en *Figuur 2*.

Wanneer we te maken hebben met opeenvolgende geheugenlocaties (= coalesced), kan de nodige data in één keer opgehaald worden, wat de uitvoeringssnelheid verhoogd. In het geval van niet-opeenvolgende geheugenlocaties moet de data in verschillende keren opgehaald worden uit het geheugen, wat de uitvoeringstijd vertraagt. Bij de RGBRGBRGB-indeling zitten de pixelwaarden niet opgeslagen op opeenvolgende adressen in het geheugen, wat resulteert in uncoalesced memory access en bijgevolg dus langere verwerkingstijden. Daarentegen staan bij de RRR...GGG...BBB-indeling de rode pixelwaarden opeenvolgend opgeslagen in het geheugen, wat parallellisme mogelijk maakt en de uitvoering versnelt.

De uitvoertijd wordt ook beïnvloed door de invoergrootte en gridgrootte, waarbij we een stijging van de uitvoeringstijd zien naarmate de invoergrootte toeneemt.

3.2 Part 2: Global, shared and constant memory

3.2.1 Comparing memory types

In dit deel vergelijken we de uitvoeringstijd van een matrixvermenigvuldiging, voor verschillende types geheugen, gebruik makend van (vierkante) matrices met verschillende invoergroottes/gridgroottes.

Wanneer er gebruik gemaakt wordt van global memory om de matrix in op te slaan, zien we dat voor een toenemende gridgrootte de tijd die nodig is om de matrices te vermenigvuldigen aanzienlijk toeneemt, dit is te zien op

Figuur 3. Global memory is een off-chip geheugen, waardoor het langer duurt om data op te halen. Daarnaast wordt er gebruik gemaakt van een L1 en een L2 cache bij het globaal geheugen. Wanneer er grotere hoeveelheden data moet worden opgehaald, treden er meer cache misses op waardoor het ophalen van data verder vertraagd voor grotere inputgroottes.

Voor shared memory zien we op *Figuur 5* een vrij constante uitvoeringstijd. Dit komt doordat Shared memory een on-chip geheugen is. Dit houdt in dat de data veel minder lang onderweg is naar de GPU, wat de uitvoeringstijd van de matrixvermenigvuldiging bevordert. Daarnaast is het ook zo dat data in shared memory opgeslagen zit in zogenaamde "memory banks" [2]. Deze kunnen parallel ge-accessed worden door verschillende threads waardoor de uitvoeringstijd, zelfs voor grotere gridgroottes, vrij constant blijft.

Voor constant memory zien we dat deze de traagste is van de drie types geheugen. Dit geheugen kan heel snel zijn en een grote tijdswinst opleveren wanneer alle threads eenzelfde waarde lezen, omwille van caching. Echter in deze use case accessed iedere thread een andere waarde waardoor er telkens cache misses optreden. Hierdoor gaat veel tijd verloren. Dit is te zien op *Figuur 6*

We zien dus dat het shared geheugen de beste uitvoeringstijd oplevert. We moeten er echter wel rekening mee houden dat het beschikbare shared geheugen beperkt is (48 kB). Het global geheugen is trager maar kan daarentegen enkele GB aan data bevatten. Hierdoor is de gridgrootte van matrices die we kunnen vermenigvuldigen via shared geheugen veel kleiner dan de gridgrootte van matrices die we kunnen vermenigvuldigen via global geheugen. Het constant geheugen kan een snel geheugen zijn, maar niet voor deze use case. Daarnaast is ook het beschikbare constant geheugen beperkt (68 kB).

3.2.2 Memory migration

Als we nu kijken naar de tijd die nodig is om data van de CPU naar het globaal geheugen van de GPU en van het globaal GPU geheugen naar het shared GPU geheugen te migreren, zien we dat het kopiëren van data van de CPU naar het globaal geheugen van de GPU langer duurt dan kopiëren van data van het globaal GPU geheugen naar het shared GPU geheugen. Dit is te zien op figuur 7. We kunnen dus stellen dat data migreren van de CPU naar de GPU meer tijd in beslag neemt dan het migreren van data tussen geheugens op de GPU. De reden hiervoor is dat het globaal geheugen en het shared geheugen beide deel uitmaken van de GPU architectuur, zie *Figuur 8*, en de CPU is een externe component die gebruik maakt van extern geheugen.

4 Conclusie

In dit labo hebben we de invloed van geheugengebruik op CUDA-kernelprestaties onderzocht. Eerst werd het verschil tussen coalesced memory access en uncoalesced memory access bekeken. Hier merkten we een snelheidswinst bij coalesced memory access ten opzichte van uncoalesced access. Dit komt doordat geheugenverzoeken gebundeld kunnen worden als de nodige data zich op opeenvolgende locaties bevindt (= coalesced access). In het tweede deel van het labo werden meerdere matrixvermenigvuldigingen uitgevoerd, gebruik makend van data die opgeslagen zit in verschillende types geheugen: globaal geheugen, gedeeld geheugen en constant geheugen. Hierbij bleek dat het gebruik van shared memory efficiënter is dan het gebruik van global memory, deze laatste is op zijn beurt dan weer sneller dan constant memory (voor deze use case). Tot slot werd nog kort ingezoomd op de tijd die nodig is om data te migreren van/naar verschillende types geheugen. We merkten dat het kopiëren van data van de CPU naar het globaal geheugen van de GPU langer duurt dan het kopiëren van data van het globaal GPU geheugen naar het shared GPU geheugen. We concluderen dus dat een doordachte geheugenstrategie cruciaal is voor CUDA-optimalisatie.

5 Graphs

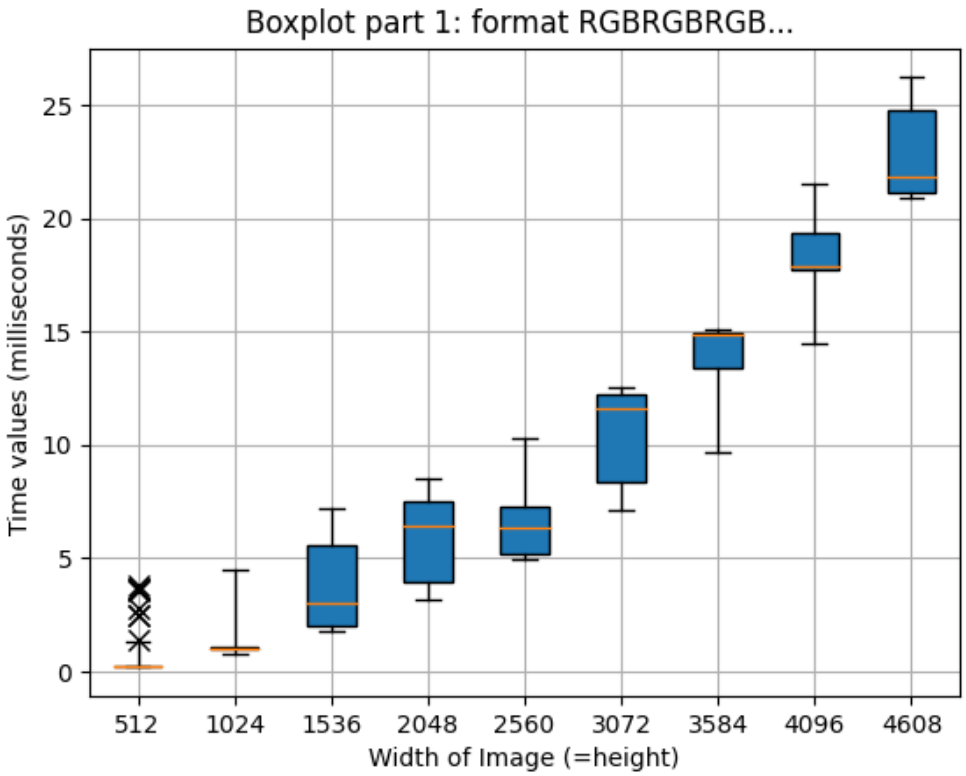


Figure 1: Boxplot part 1: format RGBRGBRGB...

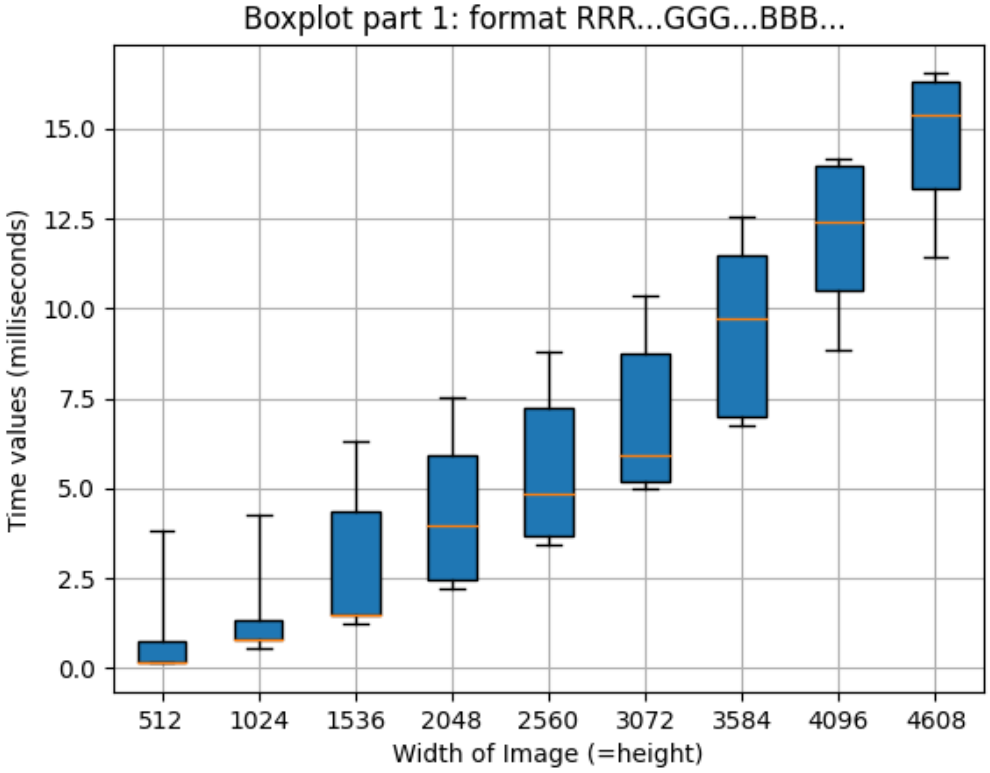


Figure 2: Boxplot part 1: format RRR...GGG...BBB...

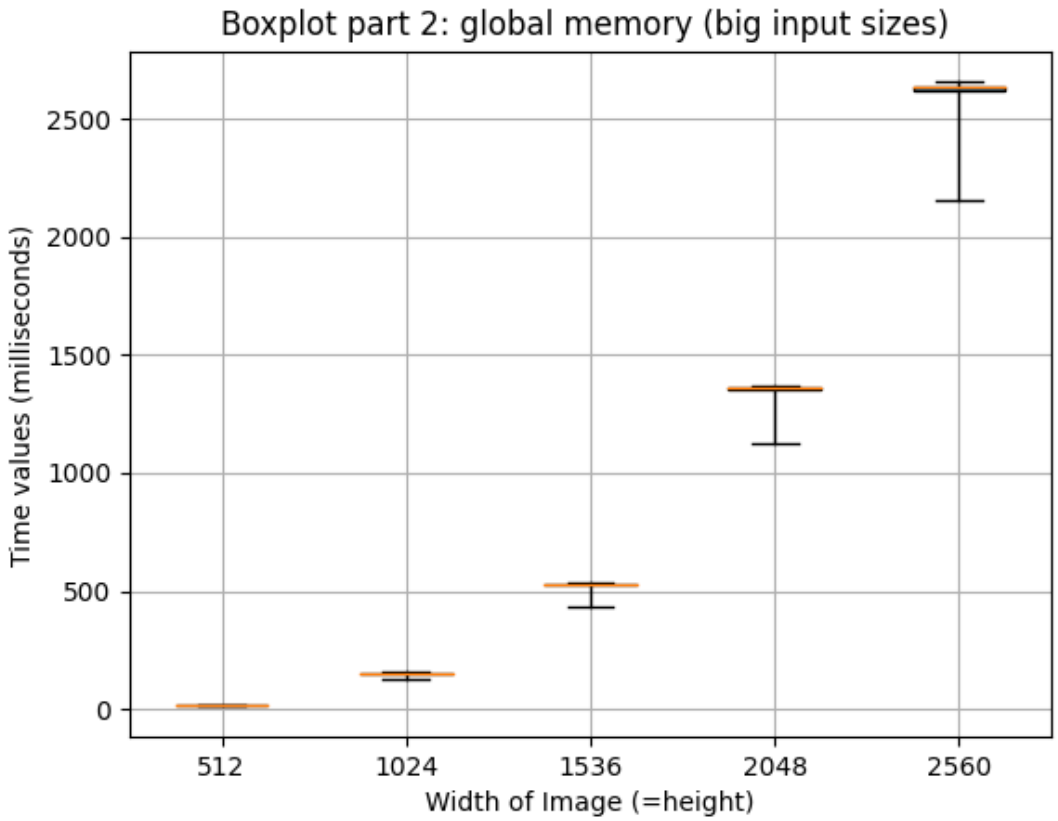


Figure 3: Boxplot part 2: global memory - big input sizes

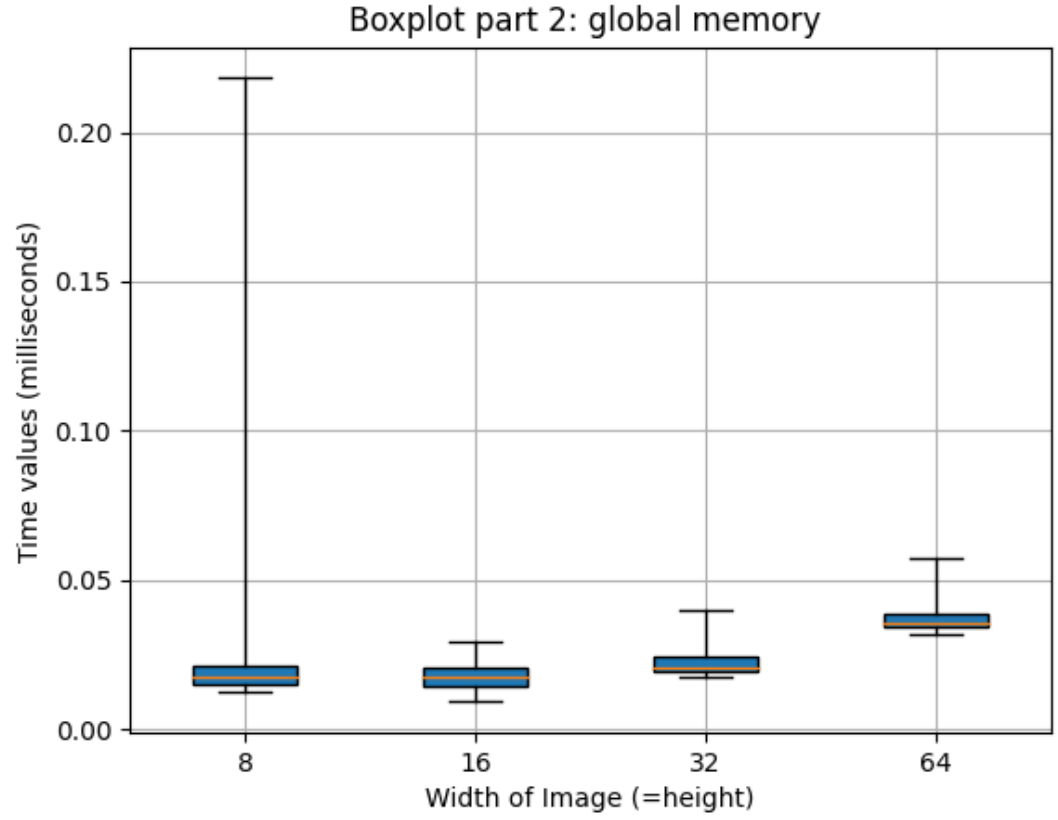


Figure 4: Boxplot part 2: global memory - small input sizes

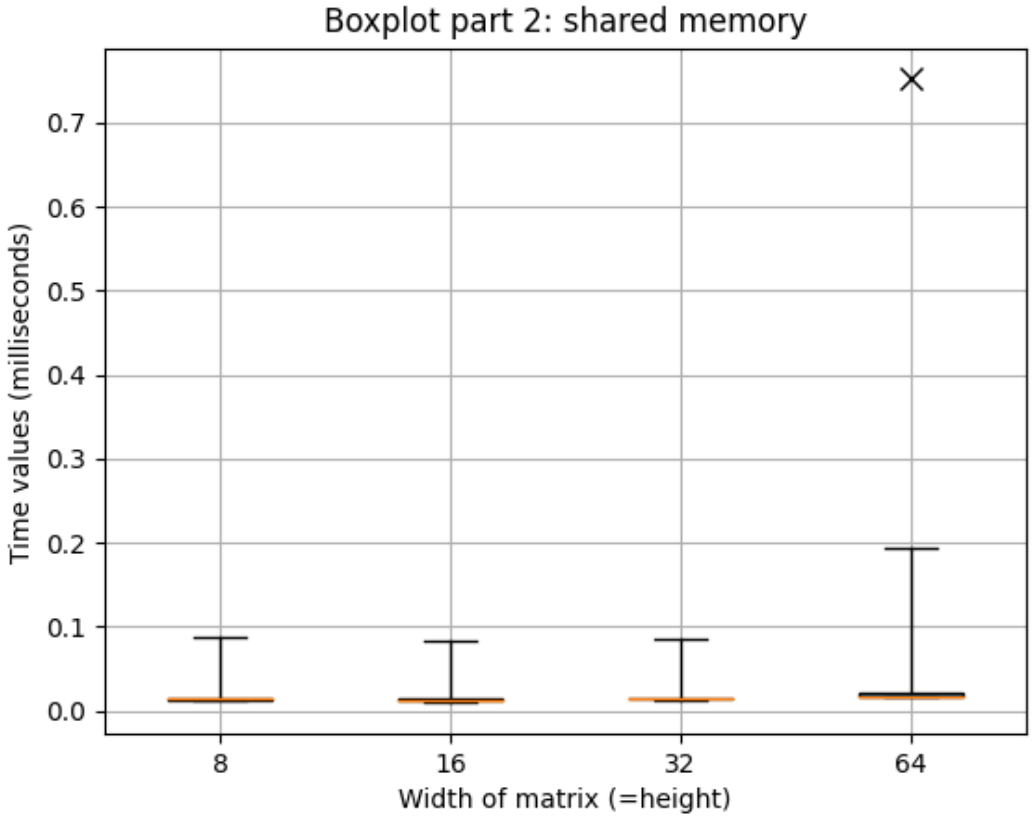


Figure 5: Boxplot part 2: shared memory

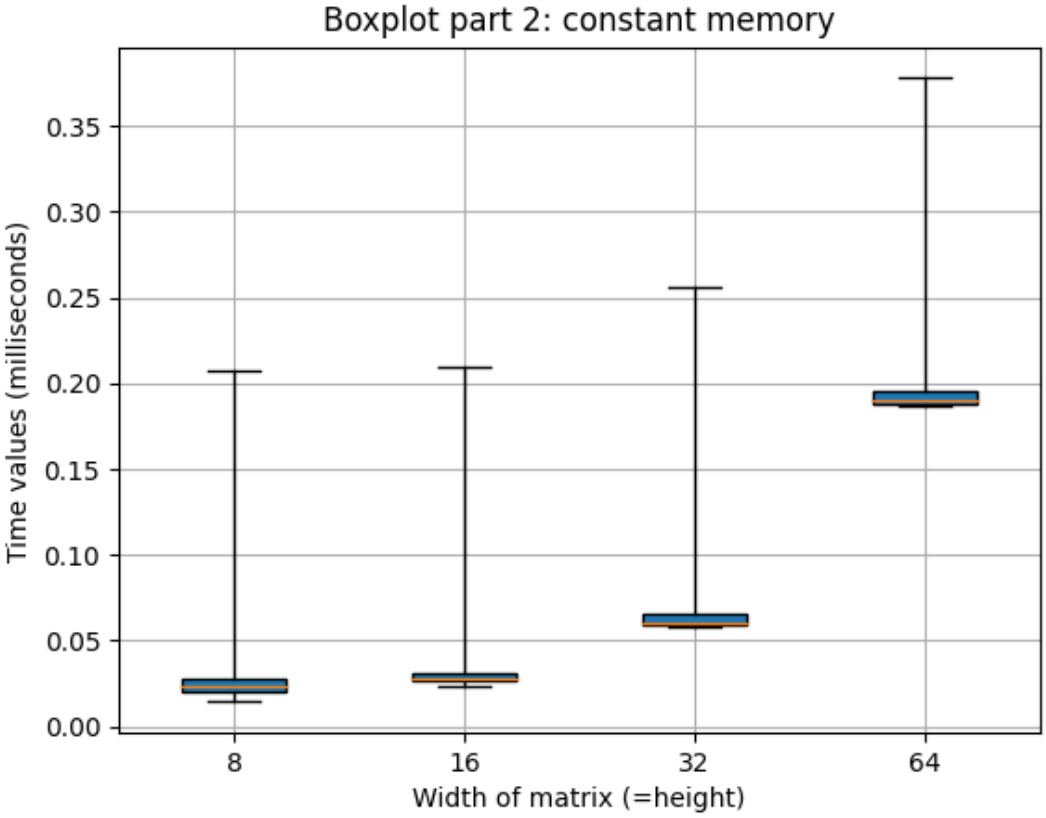


Figure 6: Boxplot part 2: constant memory

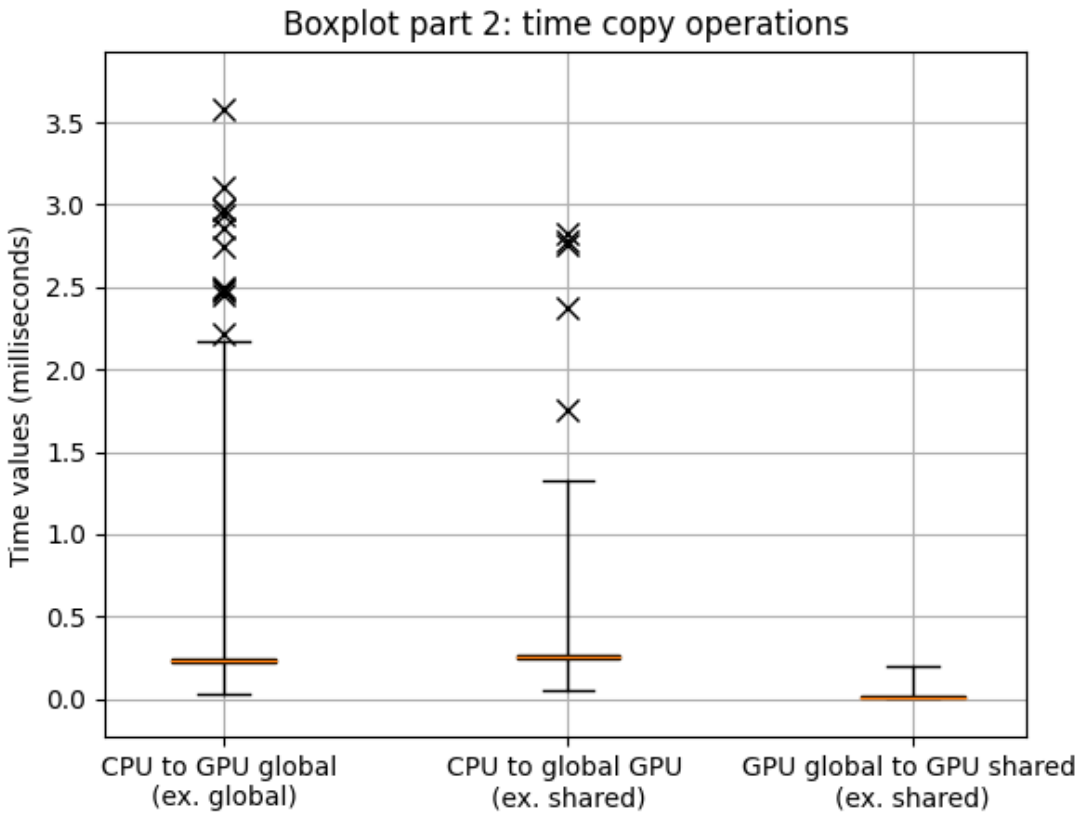


Figure 7: Memory migration time (CPU to GPU global vs GPU global to GPU shared)

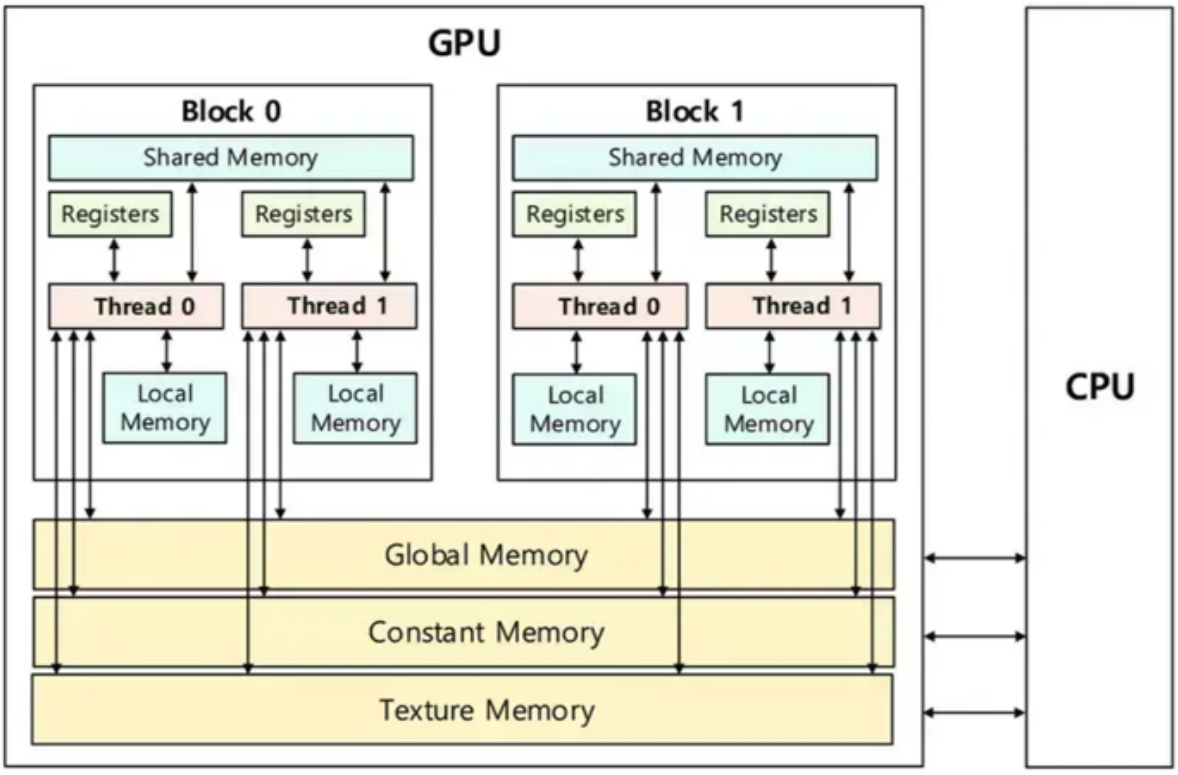


Figure 8: GPU architecture

Appendix A: Code part 1 (RGBRGB... format)

```
#include <cstdint>  // Data types
#include <iostream> // File operations

#define M 960     // VR width //960
#define N 1280    // VR height //1280
#define C 3       // Colors
#define OFFSET 16 // Header length

uint8_t *get_image_array(void)
{
    FILE *imageFile;
    imageFile = fopen("./anna.ppm" , "rb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }
    uint8_t *image_array = (uint8_t *)malloc(M * N * C * sizeof(uint8_t) + OFFSET);
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET, imageFile);
    fclose(imageFile);
    return image_array + OFFSET;
}

void save_image_array(uint8_t *image_array)
{
    FILE *imageFile;
    imageFile = fopen("./output_Image_test.ppm", "wb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }
    fprintf(imageFile, "P6\n");          // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N); // dimensions
    fprintf(imageFile, "255\n");         // Max pixel

    fwrite(image_array, 1, M * N * C, imageFile);
    fclose(imageFile);
}

__global__ void invertImage(uint8_t *image_array_d)
{
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    //Op zelfde array doen en niet invullen in nieuwe array
    //Anders moet je de andere elementen kopi ren en heb je toch coalesced memory access
    if(idx % 3 == 0){
        image_array_d[idx] = (uint8_t)255 - image_array_d[idx]; //uncoalesced
    }
    __syncthreads();
}

int main(void)
{
    uint8_t *image_array = get_image_array();
    uint8_t *image_array_d;

    cudaMalloc((void **)&image_array_d, M * N * C * sizeof(uint8_t));
    cudaMemcpy(image_array_d, image_array, M *N * C * sizeof(uint8_t), cudaMemcpyHostToDevice);

    invertImage<<<N*M*C/1024, 1024>>>(image_array_d);
    cudaDeviceSynchronize();

    cudaMemcpy(image_array, image_array_d, M * N * C * sizeof(uint8_t), cudaMemcpyDeviceToHost);
    save_image_array(image_array);
    return 0;
}
```

Appendix B: Code part 1 (RRR...GGG...BBB... format)

```
#include <cstdint>  // Data types
#include <iostream> // File operations
#define M 960      // VR width
#define N 1280     // VR height
#define C 3        // Colors
#define OFFSET 16 // Header length

uint8_t *get_image_array(void)
{
    FILE *imageFile;
    imageFile = fopen("./anna.ppm", "rb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }
    uint8_t *image_array = (uint8_t *)malloc(M * N * C * sizeof(uint8_t) + OFFSET);
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET, imageFile);
    fclose(imageFile);
    return image_array + OFFSET;
}

void save_image_array(uint8_t *image_array)
{
    FILE *imageFile;
    imageFile = fopen("./output_image_ex2_diff_format.ppm", "wb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }
    fprintf(imageFile, "P6\n");          // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N); // dimensions
    fprintf(imageFile, "255\n");         // Max pixel
    fwrite(image_array, 1, M * N * C, imageFile);
    fclose(imageFile);
}

__global__ void invertImage(uint8_t *image_array_d)
{
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if(idx < M*N*C/3){
        image_array_d[idx] = (uint8_t)255 - image_array_d[idx]; //coalesced
    }
    __syncthreads();
}

int main(void)
{
    uint8_t *image_array = get_image_array();
    uint8_t *reformat_array = (uint8_t *)malloc(M*N*C);
    int pixel_count = M*N;
    for(int i = 0; i < pixel_count; i++){
        reformat_array[i] = image_array[i * 3];
        reformat_array[i + pixel_count] = image_array[i * 3 + 1];
        reformat_array[i + 2 * pixel_count] = image_array[i * 3 + 2];
    }
    uint8_t *image_array_d;

    cudaMalloc((void **)&image_array_d, M * N * C * sizeof(uint8_t));
    cudaMemcpy(image_array_d, reformat_array, M*N*C* sizeof(uint8_t), cudaMemcpyHostToDevice);
    invertImage<<<N*M*C/1024, 1024>>>(image_array_d);
    cudaDeviceSynchronize();
    cudaMemcpy(reformat_array, image_array_d, M*N*C* sizeof(uint8_t), cudaMemcpyDeviceToHost);

    uint8_t *new_rereformat_array = (uint8_t *)malloc(M*N*C);
    for(int i = 0; i < pixel_count; i++){
        new_rereformat_array[i * 3] = reformat_array[i];
        new_rereformat_array[i *3 + 1] = reformat_array[i + pixel_count];
        new_rereformat_array[i *3 + 2] = reformat_array[i + 2 * pixel_count];
    }
    save_image_array(new_rereformat_array);
    return 0;
}
```

Appendix C: Code part 2 (Global memory)

```

#include <cstdint>  // Data types
#include <iostream> // File operations

#define height_a1 2
#define width_a1 3

#define height_a2 3
#define width_a2 2

#define height_solution 2
#define width_solution 2


__global__ void multiplyMatrices(int *array_1_d, int *array_2_d, int *new_solution_array_d)
{
    //int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int row = (blockIdx.x * blockDim.x + threadIdx.x) /width_solution;
    int col = (blockIdx.x * blockDim.x + threadIdx.x) %width_solution;

    if (row < height_solution && col < width_solution) {
        int sum = 0;
        for (int k = 0; k < width_a1; k++) {
            sum += array_1_d[row * width_a1 + k] * array_2_d[k * width_a2 + col];
        }
        new_solution_array_d[row * width_solution + col] = sum;
    }

    __syncthreads();
}

void printMatrix(int *M) {
    for (int i = 0; i < width_solution; i++) {
        for (int j = 0; j < height_solution; j++) {
            std::cout << M[i * width_solution + j] << " ";
        }
        std::cout << std::endl;
    }
}

int main(void)
{
    int array_1[width_a1 * height_a1] = {1, 2, 3, 4, 5, 6};
    int array_2[width_a2 * height_a2] = {10, 11, 20, 21, 30, 31};

    int *new_solution_array = (int *)malloc(width_solution*height_solution * sizeof(int));

    int *array_1_d;
    int *array_2_d;
    int *new_solution_array_d;

    cudaMalloc((void **)&array_1_d, width_a1 * height_a1 * sizeof(int));
    cudaMalloc((void **)&array_2_d, width_a2 * height_a2 * sizeof(int));
    cudaMalloc((void **)&new_solution_array_d, width_solution*height_solution* sizeof(int));

    cudaMemcpy(array_1_d, array_1, width_a1 * height_a1 * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(array_2_d, array_2, width_a2 * height_a2 * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(new_solution_array_d, new_solution_array, width_solution*height_solution * sizeof(
        int), cudaMemcpyHostToDevice);

    multiplyMatrices<<<1, width_solution*height_solution>>>(array_1_d,array_2_d,
        new_solution_array_d);
    cudaDeviceSynchronize();

    cudaMemcpy(new_solution_array, new_solution_array_d, width_solution*height_solution * sizeof(
        int), cudaMemcpyDeviceToHost);

    printMatrix(new_solution_array);

    cudaFree(array_1_d);
    cudaFree(array_2_d);
    cudaFree(new_solution_array_d);

    return 0;
}
```

Appendix D: Code part 2 (Shared memory)

```
#include <cstdint>  // Data types
#include <iostream> // File operations
#define n 4
#define block_size 4
#define k n/block_size
#define amount_of_blocks n*n/(block_size*block_size)

__global__ void multiplyMatrices(int *array_1_d, int *array_2_d, int *new_solution_array_d)
{
    __shared__ int array1_s[n*n];
    __shared__ int array2_s[n*n];

    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    array1_s[idx] = array_1_d[idx];
    array2_s[idx] = array_2_d[idx];

    __syncthreads();

    int row = (blockIdx.x * blockDim.x + threadIdx.x) /n;
    int col = (blockIdx.x * blockDim.x + threadIdx.x) %n;

    if (row < n && col < n) {
        int sum = 0;
        for (int i = 0; i < n; i++) {
            sum += array1_s[row * n + i] * array2_s[i * n + col];
        }
        new_solution_array_d[row * n + col] = sum;
    }
    __syncthreads();
}

void printMatrix(int *M) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            std::cout << M[i * n + j] << " ";
        }
        std::cout << std::endl;
    }
}

int main(void)
{
    int array_1[n * n] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
    int array_2[n * n] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};

    int *new_solution_array = (int *)malloc(n*n * sizeof(int));
    int *array_1_d;
    int *array_2_d;
    int *new_solution_array_d;

    cudaMalloc((void **)&array_1_d, n * n * sizeof(int));
    cudaMalloc((void **)&array_2_d, n * n * sizeof(int));
    cudaMalloc((void **)&new_solution_array_d, n*n* sizeof(int));

    cudaMemcpy(array_1_d, array_1, n * n * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(array_2_d, array_2, n * n * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(new_solution_array_d, new_solution_array, n*n * sizeof(int),
        cudaMemcpyHostToDevice);

    multiplyMatrices<<<amount_of_blocks, block_size * block_size>>>(array_1_d,array_2_d,
        new_solution_array_d);
    cudaDeviceSynchronize();

    cudaMemcpy(new_solution_array, new_solution_array_d, n*n * sizeof(int),
        cudaMemcpyDeviceToHost);

    printMatrix(new_solution_array);

    cudaFree(array_1_d);
    cudaFree(array_2_d);
    cudaFree(new_solution_array_d);

    return 0;
}
```

Appendix E: Code part 2 (Constant memory)

```

#include <cstdint>  // Data types
#include <iostream> // File operations

#define height_a1 2
#define width_a1 3

#define height_a2 3
#define width_a2 2

#define height_solution 2
#define width_solution 2

__constant__ int array_1_d[width_a1 * height_a1] ;
__constant__ int array_2_d[width_a2 * height_a2] ;

__global__ void multiplyMatrices(int *new_solution_array_d) //int *array_1_d, int *array_2_d,
{
    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    int row = idx /width_solution;
    int col = idx %width_solution;

    if (row < height_solution && col < width_solution) {
         int sum = 0;
         for (int k = 0; k < width_a1; k++) {
             sum += array_1_d[row * width_a1 + k] * array_2_d[k * width_a2 + col];
         }
         new_solution_array_d[row * width_solution + col] = sum;
    }
    __syncthreads();
}

void printMatrix(int *M) {
    for (int i = 0; i < width_solution; i++) {
        for (int j = 0; j < height_solution; j++) {
            std::cout << M[i * width_solution + j] << " ";
        }
        std::cout << std::endl;
    }
}

int main(void)
{
    int array_1[width_a1 * height_a1] = {1, 2, 3, 4, 5, 6};
    int array_2[width_a2 * height_a2] = {10, 11, 20, 21, 30, 31};

    int *new_solution_array = (int *)malloc(width_solution*height_solution * sizeof(int));
    int *new_solution_array_d;

    cudaMalloc((void **)&new_solution_array_d, width_solution*height_solution* sizeof(int));

    //Kopie data naar constant memory
    cudaMemcpyToSymbol(array_1_d, array_1, width_a1 * height_a1 * sizeof(int));
    cudaMemcpyToSymbol(array_2_d, array_2, width_a2 * height_a2 * sizeof(int));

    cudaMemcpy(new_solution_array_d, new_solution_array, width_solution*height_solution * sizeof(
        int), cudaMemcpyHostToDevice);

    multiplyMatrices<<<1, width_solution*height_solution>>>(new_solution_array_d);
    cudaDeviceSynchronize();

    cudaMemcpy(new_solution_array, new_solution_array_d, width_solution*height_solution * sizeof(
        int), cudaMemcpyDeviceToHost);

    printMatrix(new_solution_array);

    cudaFree(new_solution_array_d);
    return 0;
}
```

References

[1] 'CUDA C++ Programming Guide'. Accessed: Mar. 3, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

[2] 'CUDA:Shared memory'. Accessed: Mar. 7, 2025. [Online]. Available: https://medium.com/@fatlip/cuda-shared-memory-23cd1a0d4e39#:~:text=CUDA%20shared%20memory%20is%20a%20type%20of%20memory%20accessible%20to,a%20GPU%20architecture%20and%20configuration.